JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class ByteArrayOutputStream

java.lang.Object
    java.io.OutputStream
        java.io.ByteArrayOutputStream

All Implemented Interfaces:

Closeable, Flushable, AutoCloseable

```
public class ByteArrayOutputStream
extends OutputStream
```

This class implements an output stream in which the data is written into a byte array. The buffer automatically grows as data is written to it. The data can be retrieved using `toByteArray()` and `toString()`.

Closing a `ByteArrayOutputStream` has no effect. The methods in this class can be called after the stream has been closed without generating an `IOException`.

Since:

1.0

Field Summary

Fields

Modifier and Type	Field	Description
`protected byte[]`	`buf`	The buffer where data is stored.
`protected int`	`count`	The number of valid bytes in the buffer.

Constructor Summary

Constructors

Constructor	Description
`ByteArrayOutputStream()`	Creates a new `ByteArrayOutputStream`.
`ByteArrayOutputStream(int size)`	Creates a new `ByteArrayOutputStream`, with a buffer capacity of the specified size, in bytes.

Method Summary

All Methods	Instance Methods	

Modifier and Type	Method	Description
void	**close**()	Closing a ByteArrayOutputStream has no effect.
void	**reset**()	Resets the count field of this ByteArrayOutputStream to zero, so that all currently accumulated output in the output stream is discarded.
int	**size**()	Returns the current size of the buffer.
byte[]	**toByteArray**()	Creates a newly allocated byte array.
String	**toString**()	Converts the buffer's contents into a string decoding bytes using the default charset.
String	**toString**(int hibyte)	**Deprecated.** This method does not properly convert bytes into characters.
String	**toString** (String charsetName)	Converts the buffer's contents into a string by decoding the bytes using the named charset.
String	**toString** (Charset charset)	Converts the buffer's contents into a string by decoding the bytes using the specified charset.
void	**write**(byte[] b, int off, int len)	Writes len bytes from the specified byte array starting at offset off to

		this ByteArrayOutputStream.
void	write(int b)	Writes the specified byte to this ByteArrayOutputStream.
void	writeBytes(byte[] b)	Writes the complete contents of the specified byte array to this ByteArrayOutputStream.
void	writeTo (OutputStream out)	Writes the complete contents of this ByteArrayOutputStream to the specified output stream argument, as if by calling the output stream's write method using out.write(buf, 0, count).

Methods declared in class OutputStream

flush, nullOutputStream, write

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Field Details

buf

```
protected byte[] buf
```

The buffer where data is stored.

count

```
protected int count
```

The number of valid bytes in the buffer.

Constructor Details

ByteArrayOutputStream

```
public ByteArrayOutputStream()
```

Creates a new `ByteArrayOutputStream`. The buffer capacity is initially 32 bytes, though its size increases if necessary.

ByteArrayOutputStream

```
public ByteArrayOutputStream(int size)
```

Creates a new `ByteArrayOutputStream`, with a buffer capacity of the specified size, in bytes.

Parameters:

`size` - the initial size.

Throws:

`IllegalArgumentException` - if size is negative.

Method Details

write

```
public void write(int b)
```

Writes the specified byte to this `ByteArrayOutputStream`.

Specified by:

`write` in class `OutputStream`

Parameters:

`b` - the byte to be written.

write

```
public void write(byte[] b,
                  int off,
                  int len)
```

Writes `len` bytes from the specified byte array starting at offset `off` to this `ByteArrayOutputStream`.

Overrides:

`write` in class `OutputStream`

Parameters:

`b` - the data.

`off` - the start offset in the data.

`len` - the number of bytes to write.

Throws:

`NullPointerException` - if `b` is `null`.

`IndexOutOfBoundsException` - if `off` is negative, `len` is negative, or `len` is greater than `b.length - off`

writeBytes

```
public void writeBytes(byte[] b)
```

Writes the complete contents of the specified byte array to this `ByteArrayOutputStream`.

API Note:

This method is equivalent to `write(b, 0, b.length)`.

Parameters:

`b` - the data.

Throws:

`NullPointerException` - if `b` is `null`.

Since:

11

writeTo

```
public void writeTo(OutputStream out)
            throws IOException
```

Writes the complete contents of this `ByteArrayOutputStream` to the specified output stream argument, as if by calling the output stream's write method using `out.write(buf, 0, count)`.

Parameters:

`out` - the output stream to which to write the data.

Throws:

`NullPointerException` - if `out` is `null`.

`IOException` - if an I/O error occurs.

reset

```
public void reset()
```

Resets the `count` field of this `ByteArrayOutputStream` to zero, so that all currently accumulated output in the output stream is discarded. The output stream can be used again, reusing the already allocated buffer space.

See Also:

`ByteArrayInputStream.count`

toByteArray

```
public byte[] toByteArray()
```

Creates a newly allocated byte array. Its size is the current size of this output stream and the valid contents of the buffer have been copied into it.

Returns:

the current contents of this output stream, as a byte array.

See Also:

`size()`

size

```
public int size()
```

Returns the current size of the buffer.

Returns:

the value of the `count` field, which is the number of valid bytes in this output stream.

See Also:

`count`

toString

```
public String toString()
```

Converts the buffer's contents into a string decoding bytes using the default charset. The length of the new `String` is a function of the charset, and hence may not be equal to the size of the buffer.

This method always replaces malformed-input and unmappable-character sequences with the default replacement string for the default charset. The CharsetDecoder class should be used when more control over the decoding process is required.

Overrides:

`toString` in class `Object`

Returns:

String decoded from the buffer's contents.

Since:

1.1

See Also:

`Charset.defaultCharset()`

toString

```
public String toString(String charsetName)
                throws UnsupportedEncodingException
```

Converts the buffer's contents into a string by decoding the bytes using the named `charset`.

This method is equivalent to `#toString(charset)` that takes a `charset`.

An invocation of this method of the form

```
ByteArrayOutputStream b;
b.toString("UTF-8")
```

behaves in exactly the same way as the expression

```
ByteArrayOutputStream b;
b.toString(StandardCharsets.UTF_8)
```

Parameters:

`charsetName` - the name of a supported `charset`

Returns:

String decoded from the buffer's contents.

Throws:

`UnsupportedEncodingException` - If the named charset is not supported

Since:

1.1

toString

```
public String toString(Charset charset)
```

Converts the buffer's contents into a string by decoding the bytes using the specified `charset`. The length of the new `String` is a function of the charset, and hence may not be equal to the length of the byte array.

This method always replaces malformed-input and unmappable-character sequences with the charset's default replacement string. The `CharsetDecoder` class should be used when more control over the decoding process is required.

Parameters:

`charset` - the charset to be used to decode the `bytes`

Returns:

String decoded from the buffer's contents.

Since:

10

toString

```
@Deprecated
public String toString(int hibyte)
```

Deprecated.
This method does not properly convert bytes into characters. As of JDK 1.1, the preferred way to do this is via the `toString(String charsetName)` or `toString(Charset charset)` method, which takes an encoding-name or charset argument, or the `toString()` method, which uses the default charset.

Creates a newly allocated string. Its size is the current size of the output stream and the valid contents of the buffer have been copied into it. Each character *c* in the resulting string is constructed from the corresponding element *b* in the byte array such that:

```
    c == (char)(((hibyte & 0xff) << 8) | (b & 0xff))
```

Parameters:

`hibyte` - the high byte of each resulting Unicode character.

Returns:

the current contents of the output stream, as a string.

See Also:

`size()`, `toString(String)`, `toString()`, `Charset.defaultCharset()`

close

```
public void close()
           throws IOException
```

Closing a `ByteArrayOutputStream` has no effect. The methods in this class can be called after the stream has been closed without generating an `IOException`.

Specified by:

`close` in interface `AutoCloseable`

Specified by:

`close` in interface `Closeable`

Overrides:

`close` in class `OutputStream`

Throws:

`IOException` - if an I/O error occurs.

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.